

1

```
import pandas as pd
import numpy as np

# Reusing the sample dataset setup
data = {
    'TV': np.random.randint(0, 300, 20),
    'Radio': np.random.randint(0, 50, 20),
    'Newspaper': np.random.randint(0, 150, 20),
    'Sales': 1.5 + 0.1 * np.random.randint(0, 300, 20) + 0.2 * np.random.randint(0, 50, 20)
}
df = pd.DataFrame(data)

# 1. Isolate the feature (X) and target (y) as NumPy arrays
X = df['Radio'].values
y = df['Sales'].values

# --- Manual Sequential Data Splitting ---
def manual_sequential_split(X, y, test_size=0.3):
    # Determine the split index
    # (1 - test_size) gives the proportion for training (e.g., 0.7 for 70%)
    split_index = int(len(X) * (1 - test_size))

    # 1. Split the X data sequentially
    # X_train gets all elements UP TO the split_index
    X_train = X[:split_index]
    # X_test gets all elements FROM the split_index onwards
    X_test = X[split_index:]

    # 2. Split the y data sequentially
    y_train = y[:split_index]
    y_test = y[split_index:]

    return X_train, X_test, y_train, y_test

# Perform the sequential split (using 30% for testing)
X_train, X_test, y_train, y_test = manual_sequential_split(X, y, test_size=0.3)

print("--- Sequential Data Splitting Results ---")
print(f"Total samples: {len(X)}")
print(f"Training samples (70%): {len(X_train)}")
print(f"Testing samples (30%): {len(X_test)}")
print(f"\nX_train starts with: {X_train[:3]}")
print(f"X_test starts with: {X_test[:3]}") # This will be the value at the split index

## Function to calculate the coefficients (m and c) - USE ONLY TRAINING DATA
def calculate_coefficients(X_train, y_train):
    mean_X = np.mean(X_train)
    mean_y = np.mean(y_train)

    numerator = np.sum((X_train - mean_X) * (y_train - mean_y))
    denominator = np.sum((X_train - mean_X)**2)

    m = numerator / denominator
    c = mean_y - (m * mean_X)

    return m, c

## Function to make predictions - USE THE TEST DATA
def predict_sales(X, m, c):
    return m * X + c

## Function to calculate RMSE - USE THE TEST DATA
def calculate_rmse(y_actual, y_predicted):
    errors = y_actual - y_predicted
    mse = np.mean(errors**2)
    rmse = np.sqrt(mse)
    return rmse

# --- Execution ---
m, c = calculate_coefficients(X_train, y_train)

print("\n--- Model Training Results (on X_train) ---")
print(f"Calculated Slope (m): {m:.4f}")
print(f"Calculated Intercept (c): {c:.4f}")

y_pred_test = predict_sales(X_test, m, c)
```

```

final_rmse = calculate_rmse(y_test, y_pred_test)

print("\n--- Model Evaluation Results (on X_test) ---")
print(f"Root Mean Squared Error (RMSE) on Test Set: {final_rmse:.4f}")

--- Sequential Data Splitting Results ---
Total samples: 20
Training samples (70%): 14
Testing samples (30%): 6

X_train starts with: [44 36 24]
X_test starts with: [44 34 48]

--- Model Training Results (on X_train) ---
Calculated Slope (m): 0.0982
Calculated Intercept (c): 20.9368

--- Model Evaluation Results (on X_test) ---
Root Mean Squared Error (RMSE) on Test Set: 9.0123

```

✓ 19

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, recall_score, accuracy_score, precision_score, f1_score
N = 40
data_social = {
    'User ID': range(1, N+1),
    'Gender': np.random.choice(['Male', 'Female'], N),
    'Age': np.random.randint(18, 65, N),
    'EstimatedSalary': np.random.randint(20000, 150000, N),
    '# Target variable: 1 if Purchased, 0 if Not Purchased':
    'Purchased': np.random.choice([0, 1], N, p=[0.6, 0.4])
}
df = pd.DataFrame(data_social)

# If your dataset is a CSV file:
# df = pd.read_csv('social_network_ads.csv')

X = df.iloc[:, [1, 2, 3]].values # Gender, Age, EstimatedSalary
y = df.iloc[:, 4].values # Purchased

le = LabelEncoder()
X[:, 0] = le.fit_transform(X[:, 0])

# 3. Split the Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)

# 4. Feature Scaling (CRITICAL for SVM)
# Scaling brings all features to the same magnitude (e.g., -1 to 1 or 0 to 1).
# This prevents 'EstimatedSalary' from dominating 'Age' or 'Gender' in the distance calculations.
sc = StandardScaler()
X_train = sc.fit_transform(X_train) # Fit on training data and transform it
X_test = sc.transform(X_test) # Only transform testing data using the training fit

# kernel='linear' specifies a Linear SVM
classifier = SVC(kernel = 'linear', random_state = 0) #if kernel is RBF then put 'rbf'
classifier.fit(X_train, y_train)

print("--- Model Training Complete ---")
print(f"Model fitted using {classifier.n_features_in_} scaled features.")

# 6. Make Predictions on the Test Set
y_pred = classifier.predict(X_test)

# 7. Create the Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\n--- Confusion Matrix ---")
print(cm)

# Interpretation of the Confusion Matrix (cm):
# [[TN FP]
#  [FN TP]]
# TN (True Negatives): Correctly predicted 0 (Not Purchased)
# FP (False Positives): Incorrectly predicted 1 when actual was 0
# FN (False Negatives): Incorrectly predicted 0 when actual was 1

```

```
# TP (True Positives): Correctly predicted 1 (Purchased)

recall = recall_score(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)

print("\n--- Model Evaluation Scores ---")
print(f"Accuracy Score: {accuracy:.4f}")
print(f"Recall Score: {recall:.4f}")
print(f"Precision Score: {precision:.4f}")
print(f"F1 Score: {f1:.4f}")
```

```
--- Model Training Complete ---
Model fitted using 3 scaled features.
```

```
--- Confusion Matrix ---
[[4 1]
 [2 3]]
```

```
--- Model Evaluation Scores ---
Accuracy Score: 0.7000
Recall Score: 0.6000
Precision Score: 0.7500
F1 Score: 0.6667
```

~ 35

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
# X contains the features (sepal length, sepal width, petal length, petal width)
X = iris.data
# y contains the target categories (0, 1, 2 representing the three Iris species)
y = iris.target

print("--- Dataset Information ---")
print(f"Features (X) shape: {X.shape}")
print(f"Target (y) unique classes: {np.unique(y)}")

# 80% for training, 20% for testing (standard split)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)

print(f"Training samples: {len(X_train)}")
print(f"Testing samples: {len(X_test)}")

# criterion='gini' specifies GINI INDEX for split calculation. for Entropy use 'entropy'
# max_depth=4 limits the tree to 4 levels deep.
# min_samples_split=2 requires a node to have at least 2 samples to be split further.
classifier = DecisionTreeClassifier(
    criterion='gini',
    max_depth=4,
    min_samples_split=2,
    random_state=42 # for reproducibility
)

classifier.fit(X_train, y_train)

print("\n--- Model Training Complete ---")
print(f"Classifier trained with Gini Index and max_depth={classifier.max_depth}")

y_pred = classifier.predict(X_test)

# loss = log_loss(y_test, y_pred_proba) for log loss

# Accuracy = (Correct Predictions) / (Total Predictions)
accuracy = accuracy_score(y_test, y_pred)

# Optional: Print the comparison of actual vs predicted for a few samples
print("\n--- Prediction Comparison (First 10 Test Samples) ---")
results = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred}).head(10)
print(results)
```

```

print("\n--- Model Evaluation Score ---")
print(f"Model Accuracy: {accuracy:.4f}")

--- Dataset Information ---
Features (X) shape: (150, 4)
Target (y) unique classes: [0 1 2]
Training samples: 120
Testing samples: 30

--- Model Training Complete ---
Classifier trained with Gini Index and max_depth=4

--- Prediction Comparison (First 10 Test Samples) ---
  Actual Predicted
0       1         1
1       0         0
2       2         2
3       1         1
4       1         1
5       0         0
6       1         1
7       2         2
8       1         1
9       1         1

--- Model Evaluation Score ---
Model Accuracy: 1.0000

```

38

```

import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)

print("--- Data Split Information ---")
print(f"Training samples: {len(X_train)}")

# Standardizes features (mean=0, std=1) to ensure the optimization converges efficiently.
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# solver='lbfgs' is a common and robust choice for small datasets.
classifier = LogisticRegression(random_state = 42, solver='lbfgs', multi_class='auto')

classifier.fit(X_train, y_train)

print("\n--- Model Training Complete ---")
print("Logistic Regression model trained for 3 classes.")

y_pred = classifier.predict(X_test)

# Accuracy = (Correct Predictions) / (Total Predictions)
accuracy = accuracy_score(y_test, y_pred)

print("\n--- Model Evaluation Score ---")
print(f"Model Accuracy: {accuracy:.4f}")

--- Data Split Information ---
Training samples: 120

--- Model Training Complete ---
Logistic Regression model trained for 3 classes.

--- Model Evaluation Score ---
Model Accuracy: 1.0000
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in v
warnings.warn(

```

39

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)

print("--- Data Split Information ---")
print(f"Training samples: {len(X_train)}")

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# We choose a Decision Tree as the base model for the Bagging ensemble.
base_estimator = DecisionTreeClassifier(random_state=42)

# n_estimators=10 means we will build 10 independent Decision Trees.
# max_samples=1.0 means each tree is trained on 100% of the samples (with replacement, which is bootstrapping).
classifier = BaggingClassifier(
    estimator=base_estimator,
    n_estimators=10,
    max_samples=1.0, # Use all available samples for each bootstrap
    random_state=42
)

classifier.fit(X_train, y_train)

print("\n--- Model Training Complete ---")
print(f"Bagging Classifier trained using {classifier.n_estimators} Decision Trees.")

y_pred = classifier.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("\n--- Model Evaluation Score ---")
print(f"Bagging Classifier Accuracy: {accuracy:.4f}")

...

classifier = RandomForestClassifier(
    n_estimators=100,
    criterion='gini',
    random_state=42
)

classifier = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42
)

weak_learner = DecisionTreeClassifier(max_depth=1, random_state=42)
classifier = AdaBoostClassifier(
    estimator=weak_learner, # Using the corrected parameter name 'estimator'
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)
...

--- Data Split Information ---
Training samples: 120

--- Model Training Complete ---
Bagging Classifier trained using 10 Decision Trees.

--- Model Evaluation Score ---
Bagging Classifier Accuracy: 1.0000
```

43

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)

sc = StandardScaler()
X_train_scaled = sc.fit_transform(X_train)
X_test_scaled = sc.transform(X_test)

print("--- Data Preparation Complete ---")
print(f"Original Feature Count: {X_train_scaled.shape[1]}")

knn_baseline = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2) # p=2 specifies Euclidean distance
knn_baseline.fit(X_train_scaled, y_train)

y_pred_baseline = knn_baseline.predict(X_test_scaled)
accuracy_baseline = accuracy_score(y_test, y_pred_baseline)

print("\n--- Model 1: KNN on Original Data ---")
print(f"Baseline Accuracy (4 features): {accuracy_baseline:.4f}")

knn_pca = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
knn_pca.fit(X_train_pca, y_train)

y_pred_pca = knn_pca.predict(X_test_pca)
accuracy_pca = accuracy_score(y_test, y_pred_pca)

print("\n--- Model 2: KNN on PCA Data ---")
print(f"PCA Accuracy (2 components): {accuracy_pca:.4f}")

# n_components=2: Reduce data from 4 dimensions to 2
pca = PCA(n_components=2)

# Fit PCA ONLY on the training data and transform both sets
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

explained_variance_ratio = pca.explained_variance_ratio_.sum()

print("\n--- PCA Transformation Complete ---")
print(f"Reduced Feature Count: {X_train_pca.shape[1]}")
print(f"Variance Preserved (Cumulative): {explained_variance_ratio:.4f}")

print("\n--- Final Comparison ---")
print(f"Accuracy on 4 Original Features: {accuracy_baseline:.4f}")
print(f"Accuracy on 2 PCA Components: {accuracy_pca:.4f}")

# Determine the comparison result
if accuracy_pca >= accuracy_baseline:
    print("\nResult: PCA was highly effective, retaining similar or better accuracy with fewer features.")
else:
    print("\nResult: The reduction slightly decreased accuracy, but we achieved significant dimensionality reduction.")

--- Data Preparation Complete ---
Original Feature Count: 4

--- Model 1: KNN on Original Data ---
Baseline Accuracy (4 features): 1.0000

--- Model 2: KNN on PCA Data ---
PCA Accuracy (2 components): 0.9333

--- PCA Transformation Complete ---
Reduced Feature Count: 2
Variance Preserved (Cumulative): 0.9555

--- Final Comparison ---
```

Accuracy on 4 Original Features: 1.0000
Accuracy on 2 PCA Components: 0.9333

Result: The reduction slightly decreased accuracy, but we achieved significant dimensionality reduction.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.